

Submitted By:

Muneeb Ur Rehman F2024376357 & Hassan Ikram F2024376395

Submitted To:

Muhammad Rizwan

Basic Summary:

We used deepseek and claude for basic visuals and graphics as instructed.

We used BST for priority management and we used Arrays for missile stock and we also used vectors for other purposes instead of arrays for smooth performance and visualization, we also used queue for targets.

Problems we faced:

Using time and pulse was a big headache and we don't know how to use it also it was harder to understand which data structure was better to use for a particular problem that is when we used vector for smoothness.

Neutralization targets and stopping them was hardest another problem we faced.

For that every time the system will rebuild the BST.

We also tried to add cursor but it was bugging badly.

Output Screenshot:

D:\Games\sipar\sipar.exe

SIPAR RADAR ENGINE | PULSE: 49

-LOG: -----

X

X

[HQ]

.

.

<O>

>> STATUS: Scanning... Press [SPACE] to fire | Press E to exit

Missiles: 15 [|||||||||||||||||]

X=Missile .=Drone <O>=Friendly HQ=Base Line=Defense N=Neutralized

```

D:\Games\sipar\sipar.exe  X  +  v
-----
SIPAR RADAR ENGINE | PULSE: 36
-LOG: -----

          X
          X
        [HQ]
          .
          .

>> STATUS: Scanning... Press [SPACE] to fire | Press E to exit

                                           Missiles: 15 [||||||||||||||]

X=Missile  .=Drone  <O>=Friendly  HQ=Base  Line=Defense  N=Neutralized

```



Code:

```
#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <windows.h>
#include <ctime> #include <cmath> using namespace

std; HANDLE hConsole =
GetStdHandle(STD_OUTPUT_HANDLE);

void setPos(int x, int y) {
COORD c;
    c.X = (short)(40 + x);
    c.Y = (short)(12 - y);    SetConsoleCursorPosition(hConsole, c);
} void setColor(int color) { SetConsoleTextAttribute(hConsole, color);
} // colors and visuals are added using claude and deepseek

void hideCursor() {    CONSOLE_CURSOR_INFO
cursorInfo;    GetConsoleCursorInfo(hConsole,
&cursorInfo);    cursorInfo.bVisible = false;
SetConsoleCursorInfo(hConsole, &cursorInfo); }

void clearScreen() {    COORD
coordScreen = { 0, 0 };
    DWORD cCharsWritten;
    CONSOLE_SCREEN_BUFFER_INFO csbi;
    DWORD dwConSize;    GetConsoleScreenBufferInfo(hConsole, &csbi);    dwConSize =
csbi.dwSize.X * csbi.dwSize.Y;    FillConsoleOutputCharacter(hConsole, (TCHAR)' ',
dwConSize, coordScreen, &cCharsWritten);
    FillConsoleOutputAttribute(hConsole, csbi.wAttributes, dwConSize, coordScreen, &cCharsWritten);
    SetConsoleCursorPosition(hConsole, coordScreen);
} // ----- FRONT PAGE MENU -----

--

bool showFrontPage() {
clearScreen();
hideCursor();    int cursorX
= 0;    int cursorY = 0;
string menuOptions[3] = {

    "START MISSION",
```

```

        "INSTRUCTIONS",
"EXIT SYSTEM"    };
    int selectedOption = 0;
    int lastOption = -1;
    int lastCursorX = -1;
    int lastCursorY = -1;
    while (true) { // Only redraw if something changed
        selectedOption || lastCursorX != cursorX || lastCursorY != cursorY) { // Draw
            border (only once) if (lastOption == -1) { // Fill entire
                background with gray COORD fillPos; for (int y = 0; y < 25;
                y++) { for (int x = 0; x < 80; x++) { fillPos.X =
                x; fillPos.Y = y;
                SetConsoleCursorPosition(hConsole, fillPos); setColor(112); // Gray
                background with black text
                cout << " ";
            }
        }

        //
        Draw black border
        setColor(112);
        for (int i = 0; i < 80; i++) {
            COORD c;
            c.X = i; c.Y = 0;
            SetConsoleCursorPosition(hConsole, c);
            cout << "=";
            c.Y = 24;
            SetConsoleCursorPosition(hConsole, c);
            cout << "=";
        }
        for (int i = 1; i < 24; i++) {
            COORD c;
            c.X = 0; c.Y = i;
            SetConsoleCursorPosition(hConsole, c);
            cout << "|";
            c.X = 78;
            SetConsoleCursorPosition(hConsole, c);
            cout << "|";
        }

        // Title
        COORD titlePos;

        titlePos.X = 20; titlePos.Y = 5;
        SetConsoleCursorPosition(hConsole, titlePos);
        setColor(112); cout <<
        "+=====+";
        titlePos.Y = 6;
        SetConsoleCursorPosition(hConsole, titlePos);
        cout << "| SIPAR DEFENSE COMMAND SYSTEM |";
        titlePos.Y = 7;
        SetConsoleCursorPosition(hConsole, titlePos);
        cout << "+=====+";

        // Instructions at bottom
        COORD instrPos;
        instrPos.X = 15; instrPos.Y = 20;
        SetConsoleCursorPosition(hConsole, instrPos);
        setColor(112); cout << "Use [W/S] to navigate | Press
        [SPACE] to select";
        //

        Menu options
        if (lastOption != selectedOption || lastOption == -1) {
            for (int i = 0; i < 3; i++) { COORD optPos;
                optPos.X = 30; optPos.Y = 11 + i * 2;
                SetConsoleCursorPosition(hConsole, optPos);
                if (i == selectedOption) {
                    cout << ">>> " << menuOptions[i] << " <<<";
                } else {
                    setColor(112);
                    cout << " " << menuOptions[i] << " ";
                }
            }
        }
        lastOption = selectedOption;
        lastCursorX = cursorX;
        lastCursorY = cursorY;
        // Input handling
        if (GetAsyncKeyState('W') & 0x8000) {
            selectedOption = (selectedOption - 1 + 3) % 3;

```

```

        cursorY = max(-5, cursorY - 1);           Sleep(200);
    }
        if (GetAsyncKeyState('S') & 0x8000) {
        selectedOption = (selectedOption + 1) % 3;
        cursorY = min(10, cursorY + 1);           Sleep(200);

        }
        if (GetAsyncKeyState(VK_SPACE) & 0x8000) {
        Sleep(200);
        if (selectedOption == 0) {
        return true; // Start mission
        } else
        if (selectedOption == 1) {
        // Show
        instructions      clearScreen();
        setColor(11);      COORD
        instrDetailPos;

        instrDetailPos.X = 10; instrDetailPos.Y = 5;
        SetConsoleCursorPosition(hConsole, instrDetailPos);
        cout << "=== INSTRUCTIONS ===";           // ... more
        instruction code
        while (!GetAsyncKeyState(VK_SPACE)) Sleep(50);
        Sleep(300);      clearScreen();
        lastOption = -1;      } else if (selectedOption == 2) {
        return false; // Exit      }
        }
        Sleep(50);
    }
}

struct Target {      int id;      float x, y;      bool
isHostile;      bool destroyed;      char symbol;      int risk;
void evaluate() {      float dist = sqrt(x * x + y * y);
risk = (isHostile ? 55 : 0) + (int)(120 / (dist + 1));      }
};

// BST for priority management
class Node { public:
Target data;
Node* left;
Node* right;
Node(Target t) { data = t; left = NULL; right = NULL; }
};

class BST {
public:
Node* root;
BST() { root = NULL; }
Node* insert(Node* node, Target
t) {
if (!node) return new
Node(t);
if (t.risk < node->data.risk) node->left = insert(node->left, t);
else node->right = insert(node->right, t);      return node;      }
void insert(Target t) { root = insert(root, t); }
Node* findPriority(Node* node, char sym) {      if
(!node) return NULL;
if (node->data.isHostile && !node->data.destroyed && node->data.symbol == sym)
return node;      Node* left = findPriority(node->left, sym);      if (left)
return left;      return findPriority(node->right, sym);      }

Node* findPriority(char sym) { return findPriority(root, sym); }
void clear() { root = NULL; }
};

class SiparSystem {
private:
vector<Target> allTargets;      queue<Target>
waveQueue;      BST activeHostiles;      int missiles;
char missileBar[21];      int breaches;      vector<string>
history;      public:      SiparSystem() {      missiles
= 20;      breaches = 0;      for (int i = 0; i <
20; i++) missileBar[i] = '#';      missileBar[20] =
'\0';      }      void initWave() {      int id = 0;
for (int i = 0; i < 15; i++) {
Target t;
t.id = id++;

t.x = 40;

```

```

        t.y = (rand() % 20) - 10;
        t.isHostile = (rand() % 3 != 0);
        t.destroyed = false;
        t.symbol = t.isHostile ? (rand() % 2 ? 'X' : '.') : 'O';
        t.evaluate();
    waveQueue.push(t);
    }
    void deployWave() {
    if (!waveQueue.empty()) {
    Target t = waveQueue.front();
    waveQueue.pop();
    allTargets.push_back(t);
    }
    void rebuildBST() {
    activeHostiles.clear();
    for (auto& t :
    allTargets) {
        if (t.isHostile &&
        !t.destroyed) {
            activeHostiles.insert(t);
        }
    }
    void updatePositions() {
    for (auto& t : allTargets) {
    if (!t.destroyed) {
        if (t.isHostile) t.x -= 1;
    else if (t.symbol == 'O') t.y += (rand() % 3 - 1);
    }
    }
    void draw() {
    for (auto& t : allTargets) {
    setPos((int)t.x, (int)t.y / 2);
    if (t.destroyed) {
    setColor(8); cout << "N";
    }
    else if (t.isHostile) {
    setSymbol(12); cout << t.symbol;
    }
    else { setSymbol(10);
    cout << "<O>";
    }
    }
    void fire() {
    if
    (missiles <= 0) {
        setPos(-20, -
        12); setColor(12);
        cout << "Not
        enough stock!";
        Sleep(700);
        return;
    }
    Node* threat = activeHostiles.findPriority('X');
    if (!threat) threat = activeHostiles.findPriority('.');
    if (threat) {
        missiles--;
        missileBar[missiles]
        = ' ';
        setPos(-25, -12); setColor(14);
        cout <<
        "[DEFENSE SYSTEM] → LOCKED ON"; Sleep(400);
        setPos(-25, -
        11);
        cout << "[MISSILE LAUNCH] → IMPACT CONFIRMED";
        Sleep(400);
        setPos(-25, -10);
        cout << "[TARGET
        DESTROYED]"; Sleep(400);
        for (auto& t :
        allTargets) {
            if (t.id == threat->data.id) {
                t.isHostile = false;
                t.destroyed = true;
            }
        }
        history.push_back("Neutralized: " + to_string(t.id));
        break;
    }
    } else {
    missiles--;
    missileBar[missiles] = ' ';
    setPos(-20, -12); setColor(11);
    cout << "You targeted non-hostile target <O>";
    Sleep(700);
    }
    void monitorBase() {
    for (auto& t : allTargets) {
    if
    (t.isHostile && !t.destroyed && t.x <= -40) {
        setPos(-25, -12);
        setColor(9);
        if (t.symbol == 'X') cout << "Enemy missile hit our
        base!";
        else if (t.symbol == '.') cout << "A hostile drone entered our
        base!";
        Sleep(700);
        t.isHostile = false;
        t.destroyed = true;
    }
    breaches++;
    }
    void printHistory() {
    setPos(-38, 10); setColor(7);
    cout << "LOG: ";

    for (auto& h : history) cout << h << " | ";
    }
    void printMissiles() {
    setPos(30,
    -12); setColor(11);
    cout << "Missiles: " <<
    missiles << " ";
    cout << "[";
    for (int i = 0; i < 20; i++) cout << missileBar[i];
    cout << "]";
    }
    void drawBaseLine() {
    setColor(9);

```

```

        for (int j = -12; j <= 12; j++) {
            setPos(-40, j);
            cout << "|";
        }
    }
    void
    printManual() {
        setPos(-38, -14);
        setColor(12); cout << "X=Missile ";
        setColor(11); cout << ".=Drone ";
        setColor(10); cout << "<O>=Friendly ";
        setColor(14); cout << "HQ=Base ";
        setColor(9); cout << "Line=Defense ";
        setColor(8); cout << "N=Neutralized";
    }
    int getNeutralizedCount() {
        int count = 0;
        for (auto& t : allTargets) if (t.destroyed) count++;
        return count;
    }
    int getBreaches() {
        return breaches;
    }
    int getRemainingMissiles() {
        return missiles;
    }
};

// MAIN
int main() {
    hideCursor();
    bool startMission = showFrontPage();
    if (!startMission) {
        clearScreen();
        setColor(11);
        cout << "\n\n\tSystem shutdown. Goodbye.\n\n";
        return 0;
    }

    SiparSystem sipar;
    int frame = 0;
    Sleep(1000);
    clearScreen();
    setColor(11);
    cout << "\n\n\t[ INITIALIZING DEFENSE SYSTEMS... ]\n";
    Sleep(2000);
    sipar.initWave();
    while (frame < 80) {
        clearScreen();
        sipar.rebuildBST();
        setColor(11);
        cout << "-----\n";
        cout << "SIPAR RADAR ENGINE\n";
        cout << "PULSE: " << frame << "\n";
        cout << "-----\n";
        setPos(0, 0);
        setColor(14);
        cout << "[HQ]";
        if (frame % 10 == 0) sipar.deployWave();
        sipar.updatePositions();
        sipar.draw();
        sipar.printHistory();
        sipar.printMissiles();
        sipar.drawBaseLine();
        sipar.printManual();
        sipar.monitorBase();
        if (GetAsyncKeyState(VK_SPACE) & 0x8000) sipar.fire();
        if (GetAsyncKeyState('E') & 0x8000) break;
        setPos(-38, -10);
        setColor(8);
        cout << ">> STATUS: Scanning... Press [SPACE] to fire | Press E to exit";
        Sleep(500);
        frame++;
    }
    // End screen with mission summary
    clearScreen();
    setColor(112);
    //
    Display final statistics and mission result
    if (sipar.getBreaches() == 0) {
        cout << "MISSION SUCCESSFUL: BASE SECURED";
    } else {
        cout << "MISSION FAILED: BASE COMPROMISED";
    }
    return 0;
}

```